Perfetto Agent Skills
性能诊断体系

从性能问题到可验证根因 · RDK X5 / Embedded Linux



4
核心 Skills

2
使用方式

10
能力场景

四大核心 Skill · 独立使用与统一路由 · 十场景扩展版图

D-Robotics

目录

CONTENT

01 工具认知 Perfetto 是什么、怎么采、怎么读

02 Skill 入门 Skill 的价值、如何提问、诊断闭环

03 四个专业 Skill Memory / CPU / Scheduling / Latency

04 实战与可信度 联合诊断、 X5 真实实证、证据分级

05 扩展与总结 十场景版图、课后三步、参考链接

01

工具认知

先看仪器： Perfetto 像心电图，也像示波器

工具认知 | Skill 入门 | 四个 Skill | 实战可信 | 扩展总结

01.Perfetto： 像心电图一样给软件做「性能体检」

发现卡顿、内存异常或帧率波动时，用时间线追踪还原「异常发生的那一刻，系统做了什么」。

性能监控只告诉我们「出了问题」

Perfetto 像示波器：追踪事件时序，看清「问题在哪、前后什么关系」

① CPU 性能

CPU 忙在哪里？哪条函数路径最耗时?

② 内存行为

内存为何增长？是否在频繁 malloc / free ?

③ 线程调度

线程为什么卡住？被唤醒后还要等多久?

④ 业务时延

一次请求或一帧数据，究竟慢在哪个阶段?

⑤ I/O 与驱动

文件读写、系统调用或中断哪里有异常?

⑥ 长时 / 回归

偶发卡顿在哪里？升级版本后哪些行为变了?

体检结果：一份可以回放、放大、逐条查看的 Trace

但 Trace 只是证据，根因仍需结合源码、日志与对照实验验证。

01.Perfetto 怎么用？从症状到分析，记住四个步骤

理解一条最常见的路径：确定问题 → 在设备录制 Trace → 打开 Perfetto UI → 进一步分析与验证。

01 描述症状

确定进程 / 线程、异常时刻
及要回答的问题

02 采集 Trace

配置数据源，在 Linux 上
用 tracebox 录制

03 查看时间线

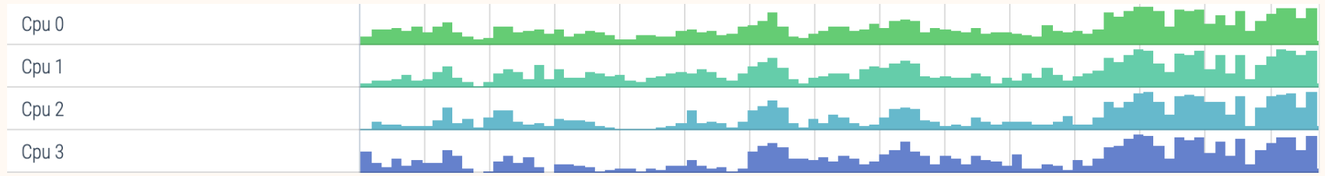
打开 .pftrace ，缩放窗口、
展开 CPU / 线程轨道

04 分析与验证

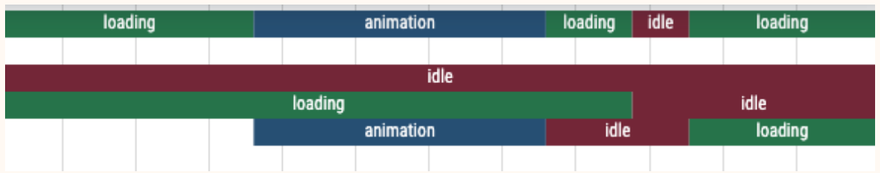
点击详情，必要时用
SQL 、源码或对照实验

真实的 Perfetto UI 长这样

CPU 活动轨道：不同 CPU 在同一时间轴上的活动



应用 Slice： 彩色区间表示被记录的一次操作 / 阶段



官方截图节选 · 两幅截图来自不同官方示例，不用于跨图推断因果。

打开后先做三件事

① 导入 访问 ui.perfetto.dev ，打开 .pftrace 或示例 Trace 。

② 找窗口 在时间线上放大异常段，展开相关线程。

③ 看证据 点击事件查看 Duration ；需统计时使用 Query （ SQL ）。

01. 为什么排障依然困难

Perfetto 是很强的采集与分析底座，但它不会替你做判断。

PART 01

症状描述模糊

「隔几秒卡一下」——不知道需要哪些证据

PART 02

工具方法分散

ftrace / perf / heapprofd / SQL 要自行组合

PART 03

结论容易越界

CPU 高≠算力瓶颈； RSS 稳≠无分配开销

要改变的是诊断方式：用户描述现象 → Agent 选择证据 → 输出可复核结论。

02

Skill 入门

把工程师用仪器的诊断流程，固化给 Agent 复用

工具认知 | Skill 入门 | 四个 Skill | 实战可信 | 扩展总结

02.Agent Skill 是什么？把诊断经验变成可执行方法

不是第二套 Perfetto，也不是一篇使用教程；它让 Agent 知道什么时候采集、如何分析、结论怎样复核。

直接使用 Perfetto

工程师需要自行决定：

01 该采哪些数据源

02 怎样生成配置并采集

03 SQL 如何写、结果如何解释

04 证据是否足以证明原因

使用 Perfetto Agent Skill

Agent 可按场景复用：

01 问题选路与诊断假设

02 TraceConfig 、采集步骤和脚本

03 经过验证的 PerfettoSQL

04 数据验收、报告和证据边界

SKILL.md

触发和选路

references / workflows

场景方法

configs / sql / scripts

可执行资产

tests / evals

验证与回归

02. 四个核心 Skill，解决四类最常见的性能问题

用户不需要先判断 Perfetto 的数据源；可以从观察到的异常现象进入。

MEMORY

内存与分配

“内存一直涨？”
“RSS 稳定但 malloc 很频繁？”

看占用、累计分配、未释放量

CPU

计算热点

“CPU 为什么这么高？”
“哪段函数在消耗 CPU？”

看运行时间与调用栈

SCHEDULING

阻塞与调度

“CPU 不高，线程为什么卡？”
“谁在唤醒它？”

看等待、唤醒和调度延迟

LATENCY

业务执行链路

“哪一帧超时？”
“处理流程慢在哪一步？”

看请求阶段、队列与端到端时延

已确定要查内存就直接调用 Memory；只知道偶尔卡顿，先从综合入口建立假设再选专业流程。

02. 第一步：像描述故障一样，直接向 Agent 提问

无需编写复杂 Prompt，更不用预先了解追踪事件、采集配置或 SQL。

用户输入示例

“RDK X5 上的 worker_app 每隔几秒就卡一下，CPU 也会波动。帮我用 Perfetto 排查一下，先设计定位方案，不要修改程序。”

如果能补充三类信息，会让 Agent 更快选对路线

01 看到什么

卡顿、CPU 波动、内存变化

02 在哪发生

RDK X5、进程名、系统环境

03 允许做什么

只给方案 / 可采集 / 可改测试程序

使用前确认 Agent 已安装或加载本项目 Skill；未授权时不直接重启、注入或修改业务。

02. 第二步： Agent 将一句话变成可复核的诊断过程

每一步都对应具体证据；如果用户只要求方案， Agent 应停留在方案交付阶段。

01

识别症状

事实与猜想分开
明确环境 / 操作边界

→

02

选择路线

建立候选假设
确定所需数据源

→

03

采集 Trace

检查板端能力
生成配置与采集

→

04

校验数据

核对目标、时间窗
丢包、缺失和符号

→

05

分析报告

SQL + UI + 调用栈
列出结论和未知

可能得到的交付物

- 诊断方案 TraceConfig
- 采集文件 .pftrace
- 分析 SQL / 证据表
- 诊断报告 待确认项

只有实际采集和查询后才能报告测量结果；缺数据时说明 UNKNOWN 与下一步补采。

02. 一套体系，两种入口：独立使用与统一路由

问题明确时直接使用专业 Skill；症状不明或跨领域时由总入口协调。

独立使用 · Direct Skill

问题范围已经明确

“这份 Trace 里谁分配了最多内存？”

↓ 直接进入 Memory Workflow

内存分配热点 / Live / Total / SQL

统一入口 · Skill Router

问题模糊或跨多个领域

“程序偶尔卡顿，帮我找原因。”

↓ 识别症状，选择一个或多个 Skill

Latency / CPU / Scheduling / Memory

两种入口都复用 Perfetto 采集、 Trace Processor 、已验证 SQL 和统一的证据要求。

注：四个独立 Skill 已开发；自动总路由为一体化目标使用形态，联调成熟度以实际验收为准。

03

四个专业 Skill

Memory / CPU / Scheduling / Latency，各看一类证据

工具认知 | Skill 入门 | 四个 Skill | 实战可信 | 扩展总结

03.Memory： 内存稳定，不代表没有分配问题

典型提问：“RSS 没涨，但程序处理时频繁 malloc / free，会不会产生开销？”

三种指标，回答三个不同问题

占用 · RSS / PSS

当前驻留多少内存?

分配 · Total Allocation

累计分配了多少?

保留 · Unreleased / Live

追踪范围内还有多少未释放?

Skill 帮你做什么

- 按增长 / 尖峰 / Churn 选路
- 采集计数器或 Native Heap
- 通过 SQL 找到分配调用路径
- 注明采样误差和未覆盖的内存

交付：内存趋势、分配热点、 Live/Total 口径和可复核 SQL；是否造成卡顿仍需 CPU / 延迟证据。

03.CPU：从“哪个线程忙”到“哪些函数在忙”

典型提问：“CPU 每隔几秒升高，具体是哪个线程、哪条调用栈在消耗资源？”

一条常见的诊断路线

1. **时间窗口** — 锁定 CPU 峰值的时刻
2. **目标线程** — 比较各线程 On-CPU 时间
3. **调用栈** — 识别采样热点与函数路径
4. **源码验证** — 核对与实际调用逻辑一致

常见输出

热点线程

谁的 CPU 时间最多

热点函数

哪个函数 / 调用路径占比高

异常窗口

热点是否只在抖动时出现

注意：CPU Skill 看 On-CPU 开销；线程等待原因由 Scheduling 进一步分析。

03.Scheduling：线程没在跑，时间究竟耗在哪里?

典型提问：“程序偶尔卡 20 ms，但 CPU 并不高，为什么线程没及时处理任务？”

一次等待的三个阶段（教学示意）

阻塞 / Sleeping

t0

等待唤醒或事件

Runnable

t1 · 唤醒

已可运行，等待获得 CPU

Running

t2 · 运行

真正执行任务

01 阻塞端调用栈

在哪一条代码路径停止推进?

02 唤醒端调用栈

谁从什么路径唤醒目标?

03 相关等待链

唤醒后又受哪些线程影响?

注意：唤醒者 ≠ 一定是锁持有者；缺失事件栈或依赖信息时，需要标记 UNKNOWN 。

03.Latency：一次请求或一帧数据，慢在哪一步?

典型提问：“视觉应用帧率波动，想知道是哪一段处理或排队造成了延迟。”

示例： Frame #1024 的业务处理链路（非实测）

Camera
输入帧

→

Queue
排队

→

CPU Pre
预处理

→

BPU Wait
提交→完成

→

Output
后处理

每个阶段可以跨线程、异步、并行； Frame ID / Request ID 才能关联同一个业务对象。

端到端时延

输入 → 输出经历多长时间

阶段与队列

时间主要消耗在哪一段

慢帧与丢帧

长尾、排队、丢弃是否出现

BPU submit 到 complete 不等于硬件纯计算时间；没有业务 ID 或关联证据就不能凭时间接近拼出一条链。

04

实战与可信度

多 Skill 协作、 X5 真实数据、证据分级

04. 一个常见卡顿问题，四个 Skill 怎么配合?

假设：“X5 的 C++ 程序 RSS 比较稳定，但每隔几秒 CPU 抖动，并伴随业务卡顿。”

先问：我们需要区分哪些可能原因？不是先认定 malloc / free 就是根因。

CPU

异常窗口谁在运行?

调用栈热点

↔

MEMORY

是不是高频分配?

Total / Churn

↔

SCHED

有没有唤醒 / 等待?

线程状态

↔

LATENCY

卡顿发生在哪段业务?

Frame / Queue

形成可信结论，需要同一异常时间窗的证据、相关调用路径和必要的对照验证。

一次诊断不一定调用全部模块；从最能区分假设的证据开始，缺什么再补。

04. 真实案例： X5 Native Heap 的 A/B 追踪结果

下面展示已经记录的真实 X5 Trace 查询结果，而不是模拟的业务性能结论。

实验 A · 无 Tick

持有期不加入周期性小分配

1 可见 dump

追踪范围内 Live 查询值 [0.0] MiB

实验 B · 200ms Tick

每 200ms 增加轻量分配推进 dump 窗口

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Dump 6
8 MiB	8 MiB	8 MiB	0 MiB	0 MiB	0 MiB

6 份可见 dump， Live 估计： 8 → 8 → 8 → 0 → 0 → 0 MiB

X5 tracebox v57.1 · Ubuntu Trace Processor v58.2 · 两份真实 .pftrace · SQL 结果可复核

证明： Heap Trace 和查询链路可验证。未证明：真实业务卡顿由 Native 分配造成。

04. 怎么判断 Agent 给出的性能诊断结果可信

一份报告必须能区分“录到了文件”“采到了目标数据”“分析方法正确”“结论证据充分”。

L1

能执行

TraceConfig 和采集命令可运行

L2

数据有效

目标、时间窗、丢包和符号可核对

L3

分析正确

SQL 统计口径与 Ground Truth 一致

L4

结论可靠

关联与因果分开，必要时做对照

“空数据 ≠ 没问题” · “时间相关 ≠ 已证明因果” · “未验证 = UNKNOWN / 需要补采”

05

扩展与总结

从四个场景到十个，再到你下一次的第一次诊断

工具认知 | Skill 入门 | 四个 Skill | 实战可信 | 扩展总结

05. 从四个核心 Skill，扩展到十个性能诊断场景

当前四个专业场景形成第一组能力；新增场景沿用相同的采集、分析与验证方法。

四个核心诊断领域

MEMORY
内存行为

CPU
执行热点

SCHEDULING
等待 / 唤醒

LATENCY
业务时延

系统级核心扩展（P1）

I/O
读写与等待

KERNEL
内核 / 驱动

POWER
频率 / 功耗

SYSTEM
跨进程竞争

扩展场景（P2）

ACCELERATOR
GPU/BPU

REGRESSION
长期追踪 / 回归

这是我们的工程场景规划，不是 Perfetto 官方的固定分类。

05. 下次遇到性能问题，可以从这三步开始

本次培训的目标不是让你成为 PerfettoSQL 专家，而是让你敢于发起一次可复核的性能诊断。

01 描述现象

哪里卡、多久一次、哪台设备或进程

02 明确执行边界

先只给方案，还是允许采集与验证

03 要求证据与下一步

保留 Trace 、 SQL 、 UNKNOWN 和复测建议

“X5 上的 C++ 程序偶发卡顿，帮我用 Perfetto 分析，先给定位方案，不要动板端。”

这句话就是一次诊断的起点。

05. 参考资料与能力边界

官方文档支撑工具能力；项目实证和测试记录支撑可验证部分的结论。

Perfetto Tracing 101	perfetto.dev/docs/tracing-101
System Tracing / UI	perfetto.dev/docs/getting-started/system-tracing
Using AI with Perfetto	perfetto.dev/docs/getting-started/using-ai
项目测试仓库	github.com/dg-xiaodong/test_perfetto

使用与验证范围

- 四类独立专业 Skill 已开发；本演示将总路由作为一体化使用形态介绍。
- 各场景的 X5 端到端成熟度以实际测试报告为准；仅 P15 展示可核验的 Native Heap 实测数据。

Thanks

留 5 分钟：你手上那个性能异常，准备怎么提问？